CSS3 Tutorial: From Zero to Confident

Written for people who already know HTML and want to learn how to style it.

Table of Contents

---

1. What CSS Actually Does

HTML describes **structure** — headings, paragraphs, lists, images. CSS describes **presentation** — colors, spacing, fonts, layout. The same HTML can look completely different depending on the CSS applied to it.

CSS works by writing **rules**. Each rule has a **selector** (what to style) and a **declaration block** (how to style it):

```
selector {
  property: value;
  property: value;
}
```

Example:

```
h1 {
  color: navy;
  font-size: 32px;
}
```

This says: "find every `<h1>` and make its text navy blue at 32 pixels."

---

2. Three Ways to Add CSS

Inline — on a single element, lowest priority for organizing real projects, but useful for quick tests:

```
<p style="color: red;">Text</p>
```

Internal — inside a `<style>` tag in your HTML `<head>`:

```
<style>
  p { color: red; }
</style>
```

External — a separate `.css` file linked from your HTML. This is the standard approach for anything beyond a quick test, because it keeps structure (HTML) and presentation (CSS) separate and lets you reuse one stylesheet across many pages:

```
<link rel="stylesheet" href="styles.css">
```

---

3. Selectors: Targeting Your HTML

You already know HTML tags, classes, and IDs — selectors just let CSS point at them.

```
p { }              /* every <p> */
.card { }          /* every element with
class="card" */
#main-nav { }      /* the one element with
id="main-nav" */
div p { }          /* every <p> that is anywhere
inside a <div> */
div > p { }        /* every <p> that is a DIRECT
child of a <div> */
h1, h2, h3 { }      /* group multiple selectors
together */
a:hover { }         /* an <a> while the mouse is
over it */
input:focus { }     /* an <input> while it's
selected */
```

```
li:first-child { }  /* an <li> that is the first
child of its parent */
li:nth-child(2n) { }/* every even <li> */
```

Specificity, in short: inline styles beat IDs, IDs beat classes, classes beat plain element selectors. If two rules conflict, the more specific one wins. When specificity ties, whichever rule comes later in the CSS wins.

---

4. The Box Model

This is the single most important concept in CSS. **Every HTML element is a rectangular box**, made of four layers, from the inside out:

```
margin
  border
    padding
      content
```

```
div {
  width: 200px;
  padding: 20px;         /* space between
content and border */
  border: 2px solid #333;/* the border itself */
  margin: 10px;           /* space outside the
border, between this box and others */
}
```

By default, `width` only sets the content area — padding and border get added on top, making the box bigger than 200px.

Most developers turn this off with:

```
* {
  box-sizing: border-box;
}
```

With `border-box`, `width` includes padding and border, so the box stays exactly the size you set. Add this at the top of every project.

---

5. Colors, Units, and Typography

Colors — several equivalent ways to write the same color:

```
color: red;
color: #ff0000;
color: rgb(255, 0, 0);
color: rgba(255, 0, 0, 0.5); /* last value is
opacity, 0–1 */
```

Units — the ones you'll use constantly:
Unit	Meaning
`px`	fixed pixels
`%`	relative to the parent element
`em`	relative to the current element's font size
`rem`	relative to the root (`<html>`) font size — predictable, good default
`vw` / `vh`	1% of viewport width / height

Typography basics:

```
body {
  font-family: "Helvetica Neue", Arial, sans-
serif;
  font-size: 16px;
  line-height: 1.5;   /* spacing between lines —
1.4–1.6 reads well */
}
h1 {
  font-weight: bold;
  text-align: center;
}
a {
  text-decoration: none; /* removes the default
underline */
}
```

---

6. Display and Positioning

Every element has a `display` type that controls how it behaves in the layout:

```
display: block;        /* takes the full width
available, stacks vertically (div, p, h1) */
display: inline;       /* only as wide as its
content, sits in a text flow (span, a) */
display: inline-block;  /* inline flow, but you
can set width/height */
display: none;          /* removed from the page
entirely */
```

`position` lets you move elements out of the normal flow:

```
position: relative; /* nudge an element from
where it would normally sit */
position: absolute;  /* place it relative to the
nearest positioned ancestor */
position: fixed;      /* stays in place even
when the page scrolls */
position: sticky;     /* acts normal, then
"sticks" once you scroll past it */
```

`top`, `left`, `right`, and `bottom` control the offset once `position` isn't `static` (the default).

---

7. Flexbox

Flexbox arranges children in a **single row or column** and is the tool you'll reach for most often — navbars, button groups, centering things.

```
.container {
  display: flex;
  flex-direction: row;          /* or column */
  justify-content: space-between; /* spacing
along the main axis */
  align-items: center;           /* alignment
along the cross axis */
  gap: 16px;                     /* space
between items, no margin hacks needed */
}
```

Try this to instantly center anything, horizontally and vertically:

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

Individual items can also control their own sizing:

```
.item {
  flex: 1; /* grow to fill available space
equally with siblings */
}
```

---

8. CSS Grid

Grid arranges children in **rows and columns at once** — best for full page layouts or anything genuinely two-dimensional (image galleries, dashboards).

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr; /* three
equal columns */
  grid-template-rows: auto;
  gap: 20px;
}
.featured {
  grid-column: span 2; /* this item takes up two
columns */
}
```

Rule of thumb: reach for Flexbox when you're arranging things in one direction; reach for Grid when you're arranging things in two.

---

9. Responsive Design

A **media query** applies CSS only when certain conditions are true, most commonly screen width:

```
/* Default: styles for all screens */
.container {
  display: flex;
  flex-direction: row;
}

/* Override for screens 768px wide or narrower
(tablets, phones) */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

Common approach: design for mobile first, then add rules for larger screens with `min-width` media queries as the layout has room to expand.

Also add this to your HTML `<head>` so mobile browsers don't zoom out to fit a desktop layout:

```
<meta name="viewport" content="width=device-
width, initial-scale=1">
```

10. Transitions and Animations

Transitions smoothly animate a property change (like a hover effect):

```
button {
  background-color: steelblue;
  transition: background-color 0.2s ease;
}
button:hover {
  background-color: darkblue;
}
```

Keyframe animations for anything more complex, like a looping effect:

```
@keyframes fade-in {
  from { opacity: 0; }
  to { opacity: 1; }
}
.card {
  animation: fade-in 0.5s ease-in;
}
```

---

11. Practice Project

Try building a simple profile card using only what's above:

1. An outer `<div class="card">` with `box-sizing: border-box`, `padding`, `border-radius`, and `box-shadow`.

2. Inside it, use **Flexbox** to arrange a profile image next to a name and short bio.
3. Add a `:hover` transition that slightly lifts the card (`transform: translateY(-4px);`) and darkens the shadow.
4. Add a media query so the layout stacks vertically (`flex-direction: column`) on screens under 480px.

If you can build that from scratch, you understand the fundamentals well enough to build real layouts.

---

12. Where to Go Next

Once this feels comfortable, look into:

- **CSS variables** (`--main-color: blue;` and `var(--main-color)`) for reusable values
- **CSS** `clamp()` for fluid, responsive font sizing without media queries
- **CSS frameworks** like Tailwind (utility classes) if you want to move faster
- **Browser DevTools** — right-click any element → Inspect — to see and live-edit CSS on real websites, which is one of the fastest ways to learn by example

---

Tip: keep this open next to your editor while you practice — you'll use the box model and Flexbox sections constantly at the start.